



**UGANDA CHRISTIAN
UNIVERSITY**

A Centre of Excellence in the Heart of Africa

Modeling and Simulating the Lake Mburo National Park Trip

Evaluating Entry Gate Queueing Delays for the 60th
Anniversary Surge

Course: SYS3205 Modeling and Simulation

Project Report

By

EZAMAMTI RONALD AUSTINE

Registration No: S23B23-018

Access No: B24252

Faculty of Engineering, Design and Technology

Department of Computing and Technology

1 Introduction

As Lake Mburo National Park approaches its 60th-anniversary celebrations, wildlife authorities are preparing for a projected 30% surge in tourist traffic. This project models the entry gate as a single-server queue and evaluates how waiting time changes as the arrival rate (λ) increases from 0.50 to 0.65 tourists per minute. The service time is held around 1.5 minutes on average, giving an effective service rate of approximately $\mu \approx 0.666$ vehicles per minute.

2 Simulation Logic

2.1 System Boundary and Conceptual Scope

The **Conceptual Model** includes the broader park visit, but the **Computational Model** isolates the entry gate as the system boundary. Activities such as game drives, boat trips, and lodge services are treated as downstream context rather than modeled explicitly. This makes it possible to focus on the gate as the primary bottleneck and compare how different service distributions affect queueing delay.

The system tracks two state variables: **server_busy**, a binary indicator for whether the ranger is serving a vehicle, and a FIFO **waiting_queue** that stores the arrival times of vehicles that must wait.

2.2 Event Handlers

- **Arrival Event:** When a vehicle arrives and the server is idle, service begins immediately and a departure event is scheduled. If the server is busy, the arrival time is stored in the waiting queue. The next arrival is always scheduled from the arrival trace so the input stream remains trace-driven.
- **Departure Event:** When a vehicle departs, the server either becomes idle or immediately begins serving the next vehicle in the queue. In the latter case, queueing delay is computed as the difference between the current clock time and the stored arrival time, and a new departure event is scheduled using the next value from the service trace.

2.3 Model Assumptions

- The entry gate is modeled as a **single-server FIFO queue**; multi-booth behavior is outside this model.
- Arrival process is Poisson, implemented through exponential inter-arrival traces for $\lambda \in \{0.50, 0.55, 0.60, 0.65\}$.
- Each experiment simulates exactly 10,000 vehicles.
- Service discipline is non-preemptive first-come-first-served.
- Service times follow one of four workload models: deterministic, exponential, hyper-exponential, or positively correlated exponential.

- System start state is empty queue and idle server at simulation time $t = 0$.

3 Implementation Description

The engine was built in standard C and follows a trace-driven simulation structure. Arrival and service values are generated in advance and read from text files during execution, which keeps the simulation reproducible and separates random variate generation from the event engine.

3.1 Trace Generation & The Inverse Transform Method

To ensure reproducibility, a standalone C module (`trace_generator.c`) pre-generates 10,000 floating-point values for each scenario. The **Inverse Transform Method** is used to convert uniform random values $U \sim (0, 1)$ into four service models:

1. Deterministic ($M/D/1$) (1.5 min baseline)
2. Exponential ($M/M/1$)
3. Hyper-Exponential ($M/G/1$) (using a 50/50 choice between two exponential means)
4. Correlated Exponential ($M/G/1$) (implemented as a simple Bernoulli-copy process that introduces positive lag-1 dependence)

During execution, the main simulator reads these external `.txt` traces dynamically. A sorted **Future Event List (FEL)** governs the chronological advancement of the event loop by always removing the earliest timestamped event.

4 Verification vs. Validation

Verification and validation were performed using the debug trace and the recorded summary results.

- **Verification (Code Checks):** The debug mode prints a manual trace of the first 10 FEL events so the event order and queue transitions can be inspected.
- **Validation (Theoretical Checks):** Using **Welford's algorithm** to compute the running mean and variance, the model produced utilization values that are close to the expected ratio $\rho = \lambda/\mu$.

$$\rho = \frac{\lambda}{\mu}$$

For example, at $\lambda = 0.50$, the deterministic service scenario reported utilization of 0.7533, which is consistent with the theoretical value given the trace-based execution.

5 Results and Discussion

The automation engine processed 16 scenarios, each based on 10,000 simulated vehicles.

Table 1: Mean and Standard Deviation of Queueing Delay vs λ

Lambda	Service Model	Mean Delay (min)	Std Dev (min)	Utilization (ρ)
0.50	Deterministic (1.5)	2.1338	2.4406	0.7533
0.50	Exponential ($M/M/1$)	4.3246	5.5887	0.7565
0.50	Hyper-Exponential	4.5686	5.7327	0.7542
0.50	Correlated Exponential	5.2802	6.7327	0.7566
0.55	Deterministic (1.5)	3.7026	4.1712	0.8274
0.55	Exponential ($M/M/1$)	7.4578	8.9106	0.8309
0.55	Hyper-Exponential	6.9403	7.7816	0.8284
0.55	Correlated Exponential	9.5466	11.5728	0.8306
0.60	Deterministic (1.5)	7.3884	8.8348	0.8962
0.60	Exponential ($M/M/1$)	13.3274	14.5896	0.9001
0.60	Hyper-Exponential	18.1951	21.7047	0.8951
0.60	Correlated Exponential	18.1036	20.5288	0.8982
0.65	Deterministic (1.5)	16.4322	13.9380	0.9638
0.65	Exponential ($M/M/1$)	46.6385	31.3576	0.9672
0.65	Hyper-Exponential	35.7642	28.7592	0.9613
0.65	Correlated Exponential	39.6212	35.5034	0.9647

5.1 Interpretation of Queueing Growth

The results show the expected queueing growth as the system approaches capacity. As traffic (λ) increases from 0.50 to 0.65, the arrival rate moves closer to the service capacity ($\mu \approx 0.66$), and the average queueing delay rises sharply.

At $\lambda = 0.60$, the deterministic service model gives the lowest mean delay among the four service patterns, while the hyper-exponential and correlated-exponential models produce similar delays of about 18.1 minutes. At $\lambda = 0.65$, the exponential model produces the highest mean delay in the table at 46.6385 minutes, showing how sensitive the system becomes near saturation.

- **Deterministic service** averaged 7.3884 minutes of delay at $\lambda = 0.60$, which is lower than the exponential and high-variability models at the same arrival rate.
- **Correlated exponential service** recorded 18.1036 minutes of delay at $\lambda = 0.60$, which is close to the hyper-exponential value of 18.1951 minutes.

6 Conclusion

The simulation indicates that the entry gate becomes increasingly congested as arrival rates approach the service capacity. In the tested scenarios, the exponential service model reached the largest mean delay, 46.6385 minutes, at $\lambda = 0.65$. A practical improvement would be to reduce service-time variability or add another server at the gate.

7 System Architecture Diagram

The figure below summarizes the project flow from trace generation through execution and output reporting.

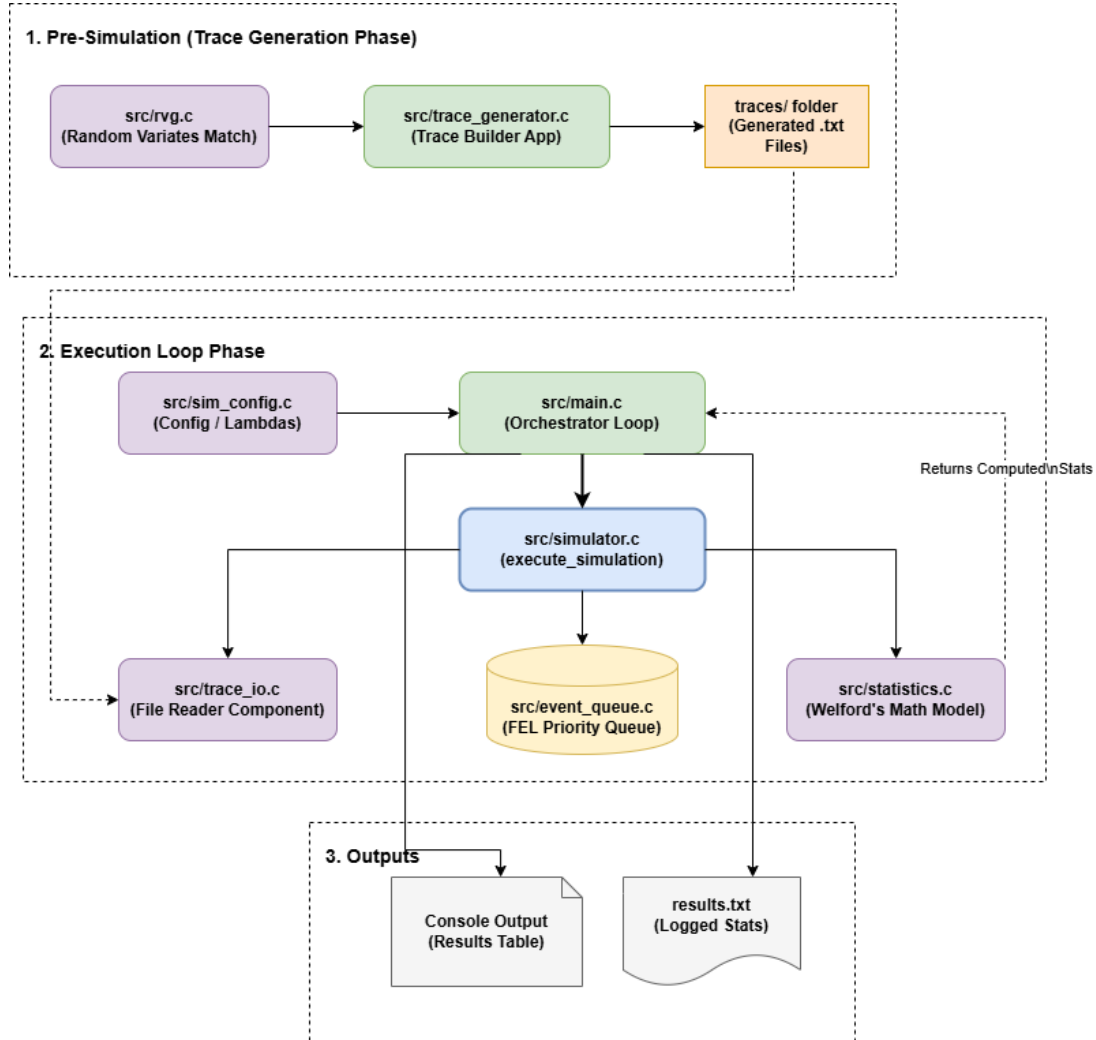


Figure 1: Overall architecture of the trace-driven simulation pipeline.